T-Detector: A Trajectory based Pre-trained Model for Game Bot Detection in MMORPGs

Sha Zhao[1*], Junwei Fang[1*], Shiwei Zhao[2], Runze Wu[2], Jianrong Tao[2], Shijian Li[1], Gang Pan[1]
[1]*Department of Computer Science, Zhejiang University, Hangzhou, China*
[2]*Fuxi AI Lab, NetEase Games, Hangzhou, China*
{szhao, fjw98218, shijianli, gpan}@zju.edu.cn {zhaoshiwei, wurunze1, hztaojianrong}@corp.netease.com

***Abstract*—Game bots are programmed to automatically play games and illegally obtain profit, seriously affecting game experience of honest players and breaking the balance of game ecosystem. Therefore, bot detection needs to be addressed urgently, especially for MMORPGs, one of the most rapidly expanding genres of games. There have been some studies for bot detection, but the features they used are dependent on specific games and the methods cannot be generalized to other games. In this paper, we propose a trajectory based pre-trained model for game bot detection from game character trajectories and mouse trajectories, named T-Detector, which is independent to specific games and can be generalized to others. More specifically, we propose a pretrain method of LocationTime2Vec to learn representations of trajectories from huge unlabeled samples, which deeply embed spatial and temporal information hidden in trajectories. Moreover, we extract universal features based on behavioral differences in movement trajectories between human players and bots. We design an Angle Pretrain to extract features of turning angle, and propose an attention pooling module to extract features of moving speed and distance. Such features are not dependent on any specific game, enabling T-Detector to be generalized to many MMORPGs. Evaluated by two large-scale real-world datasets of 143,938 samples from two MMORPGs, T-Detector achieves the state-of-the-art performance in bot detection, and demonstrates powerful generalization ability.**

***Index Terms*—game bot detection, pretrain, feature extraction**

I. Introduction

MMORPGs (massively multiplayer online role-playing game) are one of the most exciting and most rapidly expanding genres of online games. In MMORPGs, thousands of players assume the roles of characters in a fictional setting and perform exploration and adventure related tasks within a narrative [1]. In game scenarios, players use their own characters earn game-in items and virtual currency, which can be sold to others for real profit in actual currency. Taking advantage of such opportunity, game bot developers design bots to insert into games, which use illegal methods to earn profits, such as accumulating experience points, game-in items and money. Game bots are automated programs that reach the system kernel and can perform continuously tough or tedious tasks without human players' supervision [2]. In other words, a game bot can take place of human players and play games itself. Accordingly, game bot consumers can easily achieve great superiority over honest players, breaking the balance of game ecosystem. Moreover, this would lead to a less meaningful game experience of honest players, and even result in player churn [3]. The game industry has suffered serious threats from game bots, and game bot detection remains one of the most urgent problems to be addressed.

However, game bot detection is a nontrivial task, since game bots are becoming more and more human-like. There have been many ways for game bot detection, such as manual detection, traditional machine learning methods, and deep learning approaches, but they all have limitations. For example, manually labeling bots is expensive, time-consuming and ineffective, especially for MMORPGs which are with a large number of players. The traditional machine learning methods mainly depend on hand-crafted statistical features, such as how much game money earned. The correlation among different features is ignored, as well as the spatial and temporal relationship. Recently, as with the rapid development of deep learning, some studies [2], [4]–[6] have achieved good performance, but the methods are difficult to be generalized to other games. The features they used are dependent on a specific game (game-domain features), so that the method lacks the ability of generalization. Besides, such complex models run slowly in practice when they are deployed.

Besides the limitations in previous methods, there are some challenges from the perspective of data, also increasing the difficulty in bot detection. First, the historical behavior data of game players used in most previous studies is heavyweight, and the collection could be expensive, time-consuming, and intrusive. It contains many interaction records, such as accepting a mission, and killing creatures. Second, most of data samples are unlabeled. It is difficult to manually label a large scale dataset for game companies. Last but not least, the data for one sample may be missing due to temporal failure.

Fortunately, abundant historical trajectories generated when one player is playing a game, provide us a new lens for detecting game bots. Trajectories include game character trajectories (one game character's moving locations in a game map over a time period) and mouse movement trajectories. First of all, the trajectory data is much more lightweight, compared with the behavioral records. Each trajectory record contains only an (x,y) coordinate and the corresponding timestamp. Moreover, there are significant differences in trajectories between human players and bots. As we can see from a real trajectory example in Fig. 1, human players and game bots behave differently on both game character trajectories and mouse trajectories. In game character trajectories in Fig. 1 (a-1) and (a-2), when turning with a relatively large angle, game bots usually directly turn while human players prefer to turn in multiple steps with

* Eaqual contribution

2022 IEEE 38th International Conference on Data Engineering (ICDE) | 978-1-6654-0883-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICDE53745.2022.00079

2375-026X/22/$31.00 ©2022 IEEE
DOI 10.1109/ICDE53745.2022.00079

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

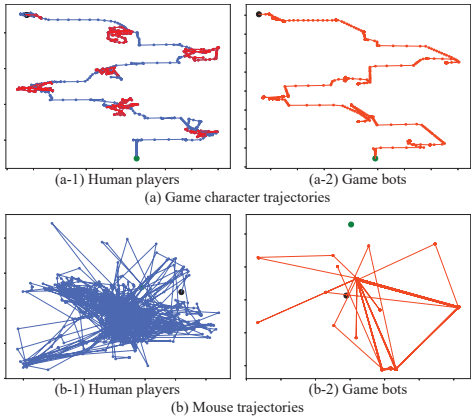

Fig. 1: A real example of one player's (human or bot) game character trajectories and mouse trajectories.

smaller angles (highlighted in red). In mouse trajectories in Fig. 1 (b-1) and (b-2), it is obvious that human players generate much more invalid movements, and the mouse trajectory looks complex. The mouse trajectories of bots are much more clear, and there are very few invalid movements. So the distance of mouse trajectories of human players is much longer than that of bots. Meanwhile, game bots are programmed to move in a relatively even speed compared with human players. Such significant behavioral differences are not dependent on any game, and can be generalized to many other games.

In this paper, in order to build a model with powerful generalization ability for bot detection, we propose a novel trajectory based pre-trained model named T-Detector. It identifies one player as a human or bot from his/her game character trajectories and mouse trajectories in a lightweight fashion. T-Detector is evaluated on a large scale dataset of 99,867 samples of Justice, a MMORPG developed by NetEase Inc, containing 87,253 human samples and 12,614 bot samples from September 30th, 2020, to October 7th, 2020. The results show its effectiveness and superiority in detection performance and running time. Moreover, T-Detector demonstrates its generalization ability on another MMORPG that is also developed by NetEase Inc. Our contributions are as follows:

- We propose a novel pre-trained model for bot detection in MMORPGs, named T-Detector, which detects game bots from players' game character trajectories and mouse trajectories. Evaluated by two large-scale real-world datasets, it achieves the state-of-the-art performance and demonstrates powerful generalization ability.
- We propose a pre-train method of LocationTime2Vec to learn representations of trajectories from huge unlabeled samples for bot detection, which deeply embed spatial and temporal information hidden in trajectories.
- We extract universal features based on the behavioral differences in movement trajectories between human players and game bots. Such features are not dependent on any specific game, enabling T-Detector to be generalized to many other MMORPGs. Specifically, we design an Angle Pretrain to extract features of turning angle, and propose an attention pooling module to extract features of moving speed and distance. In particular, turning angle features contribute significantly to bot detection performance.
- We propose a TriLoss function for training, making the model perform well even when trajectory data is missing.

II. Related Work

There have been many studies to detect bots from game player historical logs, each of which contains the player ID, action (e.g., log in), event (e.g., completing quests), and timestamp. Most players' logs can be organized as sequential data in time order, so lots of previous studies have extracted player behavioral feature sequences from historical logs to detect bots. For example, Bernardi *et al.* [7] used Multi-Layer Perceptron perceto process the sequences composed of player id, player behavior, group activity, social behaviors, etc. [6] used transformers to detect bots from the user behavior sequences composed of game events. Although these approaches based on behavioral feature sequences achieved good performance, the features they used are game-domain ones, and dependent on specific games. Thus, the approaches are difficult to be generalized to other games. Meanwhile, such game-domain features are heavyweight, of which the collection is expensive and intrusive, and the pre-processing is time-consuming [8]. Besides, billions of groundtruth samples are required for training [6]. Unfortunately, it is difficult to collect a large scale of labeled dataset, and there are a huge number of unlabeled samples.

As mentioned in the introduction, game trajectories that is lightweight can be used for classifying whether one player is bot. Many previous work using trajectory data (e.g., GPS trajectory, check-in trajectory) have similar goals as ours and motivate us a lot, for example, classifying transportation modes (bus, taxi, etc.) [9], [10], recognizing social function of regions [11], and detecting abnormality [12]. For the classification goals, at the early stage, many studies designed statistical features and applied traditional machine learning methods (e.g., SVM, Logistic Regression) for trajectory classification. Statistical features, such as velocity, acceleration, turning angle, distance, are calculated from trajectories, as well as their maximum, minimum, and average values [13]–[15]. For example, dividing the trajectories into sub-trajectories, Soleymani et al. [16] averaged the velocity and acceleration of sub-trajectories in a fixed time window. Although these features are easy to manually calculate, they are dependent on specific datasets, leading to the lack of generalization.

To address this limitation, some studies have explored deep learning techniques to automatically extract features from trajectories. Some of them take the map where trajectories are located as a graph, divide the map into grids, and apply image processing models like CNN [9], [17] for classification. For example, Zhang et al. [9] took the map as input, and transformed the trajectories into a point sequence which is mapped to a grid graph. DenseNet and attention modules were used for classifying transportation mode. Some studies regard trajectories as a sequence of trajectory points in time order, and use temporal sequence models (e.g., RNN, Transformer)

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

to model sequences. For example, longitude and latitude of trajectory points were concerted into fixed-dimensional vector, which is combined with the time embedding vector and the POI embedding vector [18]. Then, the vector sequences are input into RNN for encoding, and the output vector of the last step of RNN is used for trajectory classification. Considering game trajectories are also collected in time order, we can regard the game character trajectories and mouse trajectories as sequences of coordinates, and design neural network to work on them. Although previous studies in trajectory classification motivate us a lot, the methods cannot be adaptive for our problem. Most of them focus on where to go and passby (e.g., a district), more coarse-grained, while we pay more attention to the details (e.g., angle) in movement at each time step.

There have been some studies using game trajectories for bot detection. For example, Chen *et al.* [19], [20] analyzed avatars' movement trajectories controlled by players to determine whether the trajectory is consistent with the human player patterns. [4] used players' mouse movement trajectories for bot detection. However, the trajectory sequences used in [19], [20] are over a time period of 10 to 15 minutes to guarantee the detection accuracy, which is too long and not preferred in practice. In this work, we use both game character trajectories and mouse trajectories for bot detection. Different from previous studies, the trajectory sequence we use is only around 90 seconds in average. Moreover, we propose a pre-trained model to leverage the huge number of unlabeled samples, and use a small number of samples for training. To our best knowledge, this is the first work to explore both game character trajectories and mouse trajectories for bot detection.

III. Problem definition

Game character trajectories in game maps and mouse movement trajectories consist of only (x,y) coordinates and timestamps. Here, we treat one's game character trajectory and mouse trajectory in the same time period as a pair of sequences. In other words, one sample contains one game character trajectory and one corresponding mouse trajectory.

Definition 1 (Game Character Trajectory): A game character trajectory is defined as a location sequence of the game character that is generated over a period of time. Let $\{l_i^p\}_{i=1}^T : (l_1^p, l_2^p, \cdots, l_{T-1}^p, l_T^p)$ denote one player p's game character trajectory over time period of T. l_t^p refers to player $p's$ game character's location at t. Each location is an (x,y) coordinate. The time interval between adjacent sampling points is approximately 400 ms in average.

Definition 2 (Mouse Trajectory): A mouse trajectory is defined as a sequence of locations of the player's mouse movement over a period of time. $\{m_i^p\}_{i=1}^T : (m_1^p, m_2^p, \cdots, m_{T-1}^p, m_T^p)$ denote player $p's$ mouse trajectory over T. m_t^p is the mouse's location at t. In line with game character trajectories, each location is also an (x,y) coordinate.

Problem (Game Bot Detection): Given one player p's game character trajectories $\{l_i^p\}_{i=1}^T$ and mouse trajectories $\{m_i^p\}_{i=1}^T$ in the same period of T, detect whether p is a game bot or not.

In order to detect game bots from game character trajectories and mouse trajectories, it is necessary to model the representation of trajectories of labeled samples. However, it is difficult for game companies to manually label a large number of game bots in practice. To leverage the huge unlabeled trajectory samples, we can first apply pretraining to learn good representations from them and then use the representations for bot detection. Pretraining on large scale datasets can learn universal representations and provide a better model initialization [21], so as to improve the performance of bot detection. There have been some studies using pre-trained embedding + RNN and CNN for solving sequence-based problems [22], [23]. Pretraining methods embed the discrete input into representation vectors and neural networks (e.g. RNN, CNN) are used to extract complex features from the representations. However, if we directly use such models for bot detection, there are still some limitations:

- *The location and time interval information in trajectories is ignored.* When using previous embedding pretrain methods like word2vec [24] for embedding pretrain, the process of converting each (x,y) coordinate to a token ignores the original location. The setting of fixed window size does not consider the time interval either. Location and time interval is helpful for bot detection, reflecting how one player moves in a game scenario.
- *The behavioral differences in trajectories between human players and bots can be hardly captured by previous studies.* As we can see from Fig. 1, there are significant differences in turning angles, moving speed and distance. Such differences are not dependent on any specific game, which help improve the generalization ability of models.
- *The correlation between game character trajectories and mouse trajectories is not considered.* Previous methods only take one type of trajectory into consideration, while there are two types of trajectories in our dataset. Moreover, the game character trajectories are generally controlled by the mouse, so they are correlated with each other. Modeling such correlation would help detect bots.

IV. T-Detector

A. Overview

We propose a novel framework named T-Detector to detect game bots from game character trajectories and mouse trajectories, which considers not only the spatial and temporal relationship in trajectories, but also the behavioral differences in turning angles, moving speed, and distance between human players and bots. As shown in Fig. 2, T-Detector consists of two parts: Pre-training and Training&Inference. In the pre-training part, we model the representation of game character trajectories and mouse trajectories by proposing a pretraining method of LocationTime2Vec to embed the spatial and temporal information of trajectories. Then, using the pre-trained embedding matrices as initial weights, we capture the temporal relationship in each type of trajectory with a ConvGRU encoder. Furthermore, we learn the physical behavioral differences in movement directions between human players

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

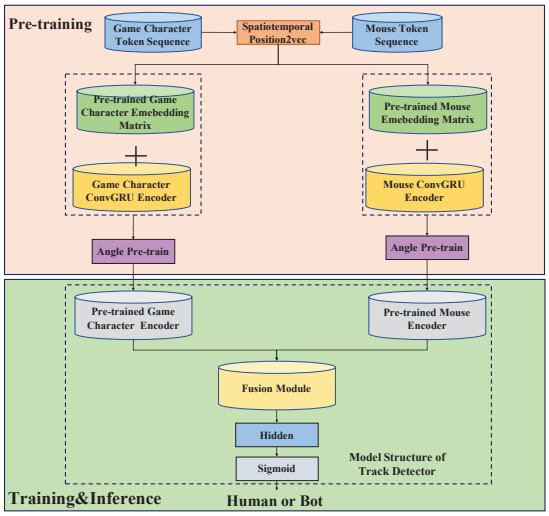

Fig. 2: T-Detector: a game bot detection framework

and bots by proposing a Angle Pretrain method to teach the ConvGRU encoders to extract turning angle features.

In Training &Inference, the ConvGRU encoders are fine-tuned. We learn the behavioral differences in moving speed and distance between human players and bots by designing an Attention Pooling module. We also model the correlation between the game character trajectories and mouse trajectories applying a Mutual Attention module. All the learned representation vectors in each module are fused together to compute the probability of being a bot with a sigmoid function.

B. *Preprocessing: dividing trajectories into grids*

We firstly perform a preprocessing to divide trajectories into grids by proposing a Merge&Split Division method. In order to embed the trajectories, we need to assign the (x,y) coordinate at each time step an index that is projected to the embedding vector. A naive approach is to divide the trajectory map into several rectangular grids of the same size, and assign each grid an index. The (x,y) coordinates falling in the same grid are assigned the same index, so that they have the same embedding vector. In this way, however, it would lead to imbalance issue of data. There would be significant differences in the number of embedding vectors, because the number of (x,y) coordinates significantly differs in each grid. In the trajectory maps, we found that players prefer to move frequently in a small number of grids. In other words, a few grids contain a large number of coordinates, while most grids contain very few coordinates.

To avoid the imbalance issue, we propose a Merge&Split Division method to divide the trajectory maps into grids. Specifically, we firstly divide each map into several 200×200 grids, and then calculate the number of (x,y) coordinates falling in each grid. We introduce a upper threshold γ and a lower threshold β, to control the number of (x,y) coordinates in each grid. When the number of coordinates in one $x \times x$ grid exceeds γ, the grid will be divided into four $x/2 \times x/2$ sub-grids, and this operation will be carried out iteratively on each sub-grid until the the number of coordinates in each sub-grid is less than or equal to γ. When the number of coordinates in

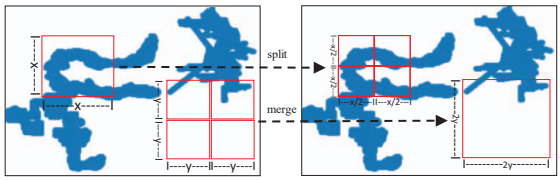

Fig. 3: Merge&Split grid division

one $y \times y$ grid is less than β, this grid and its three adjacent $y \times y$ grids will be merged to form a new $2y \times 2y$ grid. This process is shown in Fig. 3. In practice, we adjusted γ and β by comparing the standard deviations of the number of effective coordinates in all grids under different γ and β values, and selected γ and β with the lowest standard deviation. For game character trajectories, we set γ as 2000 and β as 20, and for mouse trajectories, we set γ as 100 and β as 2.

C. *Pretraining: modeling the representation of trajectories*

The Pretraining part of T-Detector consists of Location-Time2Vec, Angle Pretrain, and ConvGRU encoders, shown in Fig. 4. In particular, the pretrain method of LocationTime2Vec captures the spatial and temporal information in trajectories. Angle Pretrain learns the behavioral differences in turning angles in movement between human players and bots.

*1) **LocationTime2Vec–embedding temporal and spatial information**:* The LocationTime2Vec is a pretraining method to address the embedding of spatial and temporal information in trajectories. As an example shown in Fig. 5, the LocationTime2Vec consists of temporal embedding and spatial embedding. In Fig. 5, each column represents a step of a trajectory. The top two rows are the time and spatial coordinates of each step. For example, (8/29 23:59:59.300) represents the date and detailed time stamp, and (368,143) is the x and y coordinate, and each row below the black line is the embedding representation of the trajectory. For each step, the final spatiotemporal embedding is the sum of temporal embedding and spatial embedding.

(a) Temporal embedding: The temporal information of trajectories is helpful for bot detection, since time spent playing is important as associated to specific day phases [25]. For example, the number of human players increases on weekends, and the percentage of bots increases around midnight especially when most human players fall asleep. Thus, we try to capture the temporal information into the embedding matrix. In particular, we divide each day into 8 time slots, and propose a time slot embedding matrix $M_{\text{ts}} \in \mathbb{R}^{8\times d}$, as well as a weekend embedding matrix $M_w \in \mathbb{R}^{2\times d}$. In addition, we also have a grid embedding matrix $M_g \in \mathbb{R}^{|\mathcal{G}|\times d}$. Based on the time stamp, we sum the location's grid embedding v^g, weekend embedding v^w and time slot embedding v^{ts} into one as our temporal embedding v^t: $v^t = v^g + v^w + v^{ts}$.

Here, we apply an improved CBOW method to learn the temporal embedding v^t. We introduce the time interval between locations as weight to insert into CBOW, motivated by the idea of time-interval Event2vec [2]. The contribution of any context location to the target location is inversely

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

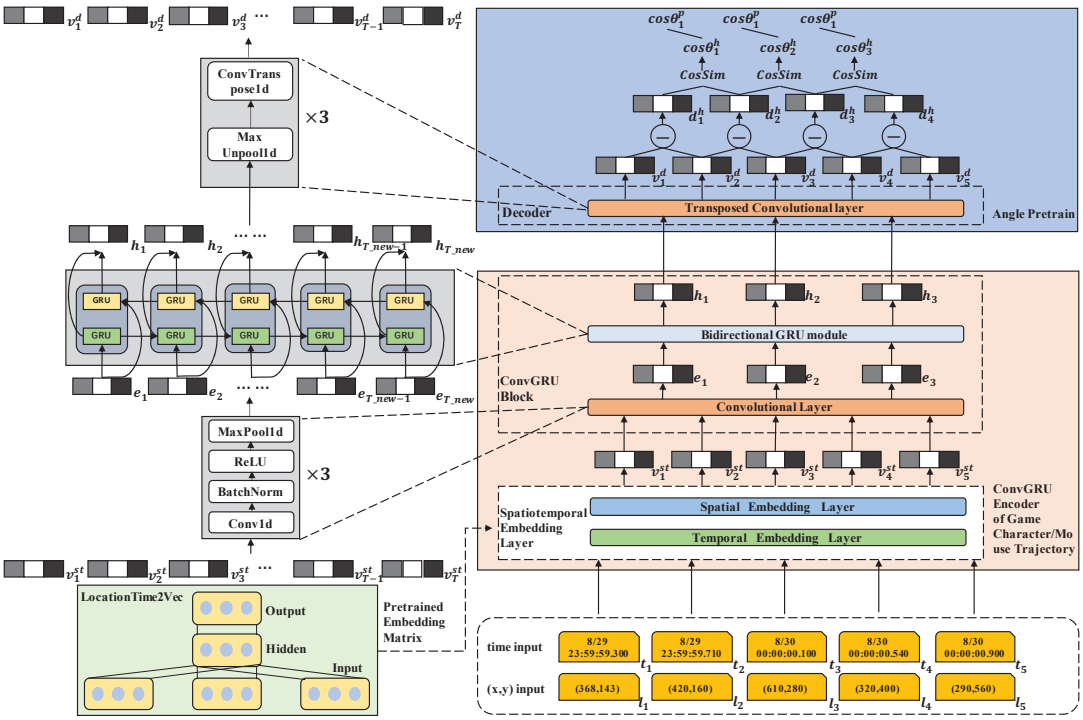

Fig. 4: Pretraining: LocationTime2Vec, ConvGRU, and Angle Pretrain.

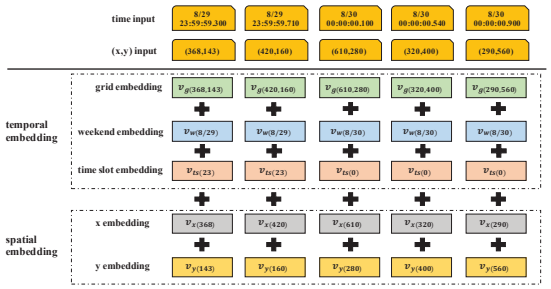

Fig. 5: LocationTime2Vec: temporal embedding and spatial embedding.

proportional to the time interval between them. We define the weight of each location p_i to the target location p_t as:

$$w\left(p_i, p_t\right) = \alpha^{l|400} \tag{1}$$

where α is set to 0.8 according to [26], and l is the time interval in millisecond between two locations. Since the time interval between most of adjacent locations is around 400ms, we use l divided by 400 as the exponent. Inserting the weight into CBOW, we get the average vector $\overline{v}$ by Equation (2). We use $\overline{v}$, and the target vector v to do the CBOW training to encode temporal context information.

$$\overline{v} = \frac{\sum_{(-c)\leq i\leq(c), i\neq 0} w\left(p_i, p_t\right) v_i}{\sum_{(-c)\leq i\leq(c), i\neq 0} w\left(p_i, p_t\right)} \tag{2}$$

where c is the context window, and v_i is the ith context vector.

(b) Spatial embedding: There are significant differences in trajectories between human players and bots, as shown in Fig. 1. For example, compared with bots, human players generate much more uncliked and invalid mouse movements. Therefore, it is important to embed the spatial information hidden in the (x,y) coordinates. In the process of temporal embedding, we embed the grid index rather than the (x, y) coordinates. Here, we design a spatial embedding module to convert the values of x and y coordinates into x vectors v^{x} and y vectors v^{y} respectively. We refer to the Learned Positional embedding [27], and learn embedding matrices $M_{\mathrm{x}} \in \mathbb{R}^{|\mathcal{X}|\times d}$ and $M_{\mathrm{y}} \in \mathbb{R}^{|\mathcal{Y}|\times d}$ for x and y respectively. The spatial embedding v^{s} is obtained by summing v^{x} and v^{y}. We skip the pretraining process for spatial embedding and fine-tune it directly.

The final spatiotemporal embedding v^{st} is the sum of v^{s} and v^{t}. The pre-trained temporal embedding matrices ($M_g \in \mathbb{R}^{|\mathcal{G}|\times d}$, $M_{\mathrm{ts}} \in \mathbb{R}^{8\times d}$, and $M_w \in \mathbb{R}^{2\times d}$) and the spatial embedding matrices ($M_{\mathrm{x}} \in \mathbb{R}^{|\mathcal{X}|\times d}$ and $M_{\mathrm{y}} \in \mathbb{R}^{|\mathcal{Y}|\times d}$) are used as the initial parameters of the embedding layers of the ConvGRU encoders below.

*2) **ConvGRU Encoder–capturing the sequential relationship in trajectories**:* Initialized by the pre-trained embedding matrices, the ConvGRU encoder learns compressed representation of game character trajectories and mouse trajectories, consisting of a Convolutional module and a Bidirectional GRU module, shown in Fig. 4. The original trajectory $(l_1, l_2, \cdots, l_{T-1}, l_T)$ and the corresponding timestamp sequence $(t_1, t_2, \cdots, t_{T-1}, t_T)$ are firstly converted into the hidden state sequence $(v_1^{st}, v_2^{st}, \cdots, v_{T-1}^{st}, v_T^{st})$ by Spatiotemporal embedding Layer, which is then input to the ConvGRU block. The convolutional operation in the Convolutional module shortens the sequence length and captures the relationship between the time steps that are far apart. The original sequence is relatively long of 90 seconds, containing about 200 sampling points (the sampling frequency is 400 ms in average). Some

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

movements may be invalid in short time period, especially in mouse trajectories. An effective movement usually occurs after multiple time steps. Here, the Convolutional module is stacked with three layers of 1-D CNN and 1-D maxpooling.

After the Convolutional module, the compressed sequence $(\mathbf{e}_1, \mathbf{e}_2, \cdots, \mathbf{e}_{T_s-1}, \mathbf{e}_{T_s})$ is input into the bidirectional GRU to model the sequential relationship among trajectories, and the encoded sequence $(\mathbf{h}_1, \mathbf{h}_2, \cdots, \mathbf{h}_{T_s-1}, \mathbf{h}_{T_s})$ is obtained.

*3) **Angle pretrain–extracting turning angle features**:* As described in Fig. 1, human players and bots behave differently when making large angle turns, especially in game character trajectories. Therefore, extracting such differences in turning angles benefits to bot detection. The mathematical procedure of calculating the turning angle of adjacent time steps using the original trajectories $(l_1^p, l_2^p, \cdots, l_{T-1}^p, l_T^p)$ is shown in Equation (3). We first calculate the direction of movement d_t^p according to adjacent coordinates and get direction vectors (dx_t^p, dy_t^p). Then the turning angle θ_t^p of two adjacent direction vectors is calculated according to the cosine theorem.

$$\begin{aligned} l_t^p &= (x_t^p, y_t^p) \quad t = 1, 2, \ldots, T \\ d_t^p &= \left(x_{t+1}^p - x_t^p, y_{t+1}^p - y_t^p\right) \\ &= (dx_t^p, dy_t^p) \quad t = 1, 2, \ldots, T-1 \\ \theta_t^p &= \arccos\left(\frac{d_t^p \cdot d_{t+1}^p}{\|d_t^p\| \left\|d_{t+1}^p\right\|}\right) \quad t = 1, 2, \ldots, T-2 \end{aligned} \tag{3}$$

Although the turning angle is important for bot detection, the ConvGRU block mentioned above hardly learn the complex feature of turning angle. To teach the ConvGRU encoders to have the ability of extracting the turning angle features, we design Angle Pretrain, which aims to fit the turning angle sequence $(\theta_1, \theta_2, \ldots, \theta_{T-1}, \theta_T)$. More specifically, motivated by the pretraining framework of AutoEncoder [28], we firstly add a Transposed Convolutional layer on our ConvGRU Encoder as the decoder, and decode the output encoded sequence $(\mathbf{h}_1, \mathbf{h}_2, \cdots, \mathbf{h}_{T_s-1}, \mathbf{h}_{T_s})$ into a decoded representation sequence $(v_1^d, v_2^d, \cdots, v_{T-1}^d, v_T^d)$ with the same length as the original sequence.

Since each time step in our original sequence corresponds to a coordinate in the trajectory, we assume that the decoded representation sequence also has the same correspondence. We expect the hidden state representing the coordinates of the corresponding step to have the same computational properties as the original (x,y) coordinates. Similar as Equation (3) which uses a sequence of coordinates to calculate the turning angle, we refer to this calculation in the Angle Pretrain and do a similar subtraction and inner product operation using the decoded representation sequence to get $\cos\theta_t^h$, The calculation is shown in follow:

$$\begin{aligned} d_t^h &= v_{t+1}^d - v_t^d \quad t = 1, 2, \ldots, T-1 \\ \cos\theta_t^h &= \left(\frac{d_t^h \cdot d_{t+1}^h}{\|d_t^h\| \left\|d_{t+1}^h\right\|}\right) \quad t = 1, 2, \ldots, T-2 \end{aligned} \tag{4}$$

We fit $\cos\theta_t^h$ to $\cos\theta_t^p$ where θ_t^p is not equal to zero. The loss function is:

$$\begin{aligned} l &= \frac{1}{|D|} \sum_{i=1}^{|D|} \sum_{\theta_t^p \neq 0}^{T_i-2} \mathrm{smooth}_{L_1}\left(\cos\theta_t^h - \cos\theta_t^p\right) \\ \mathrm{smooth}_{L_1}(x) &= \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{oterwise } j \end{cases} \end{aligned} \tag{5}$$

where smooth L1 loss [29] is used to measure the differences between model outputs and labels.

So far, all the pretraining tasks are completed (Location-Time2Vec and Angle Pretrain), as shown in Fig. 4. The ConvGRU Encoders of two types of trajectories will be fine-tuned in the part of Training&Inference.

D. *Training & Inference*

In this part, the ConvGRU encoders will be fine-tuned. Moreover, we design a Dis&Sim Attention Pooling module to extract other behavioral features in movement: speed and distance. Meanwhile, considering the correlation between game character trajectories and mouse trajectories, we apply a mutual attention mechanism to deeply integrate the two types of trajectories. Finally, the probability of being a bot is computed. The procedure is shown in Fig. 6.

*1) **Dis&Sim Attention Pooling–extracting features of moving speed and distance**:* There are significant differences in moving speed and distance between human players and bots. As we can see from Fig. 1, the moving distance in total of bots is much shorter than that of human players, especially in mouse trajectories. Besides, compared with human players, the bots are programmed to move in a relatively even speed. Such behavioral differences help detect the bots. Therefore, we design an attention pooling module to extract the speed and distance features through the operations as follows:

$$\begin{aligned} \mathbf{d}_i &= \mathbf{h}_{i+1} - \mathbf{h}_i \quad i = 1, 2, \ldots, T_s - 1 \\ \mathbf{s}_i &= \mathbf{h}_{i+1} \odot \mathbf{h}_i \quad i = 1, 2, \ldots, T_s - 1 \\ a_i &= \tanh\left(\mathbf{w} \cdot [\mathbf{d}_i : \mathbf{s}_i]\right) \quad i = 1, 2, \ldots, T_s - 1 \\ \alpha_i &= \frac{exp(a_i)}{\sum_k^{T_s-1} exp(a_k)} i = 1, 2, \ldots, T_s - 1 \\ \mathbf{h}_a &= \sum_i^{T_s-1} \alpha_i a_i \end{aligned} \tag{6}$$

By Equation (6), we obtain the distance sequence $(\mathbf{d}_1, \mathbf{d}_2, \cdots, \mathbf{d}_{T_s-1})$ by a subtraction operation on the output sequence of the encoder $(\mathbf{h}_1, \mathbf{h}_2, \cdots, \mathbf{h}_{T_s-1}, \mathbf{h}_{T_s})$, and get the speed sequence $(\mathbf{s}_1, \mathbf{s}_2, \cdots, \mathbf{s}_{T_s-1})$ by Hadamard product, respectively. The distance sequence and the speed sequence actually reflect the similarity and difference of coordinates, which can be related to the speed and distance features. For instance, if the similarity is relatively small, it indicates the moving distance is relatively long in the same time period, so the speed is relatively quick. Thus, we assume that the concatenated and transformed sequence $(\mathbf{a}_1, \mathbf{a}_2, \cdots, \mathbf{a}_{T_s-1})$ could reflect features of speed and distance, and it is finally aggregated as a fixed dimension vectors $\mathbf{h}_a$ through Attention pooling. The procedure is shown in Fig. 7.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

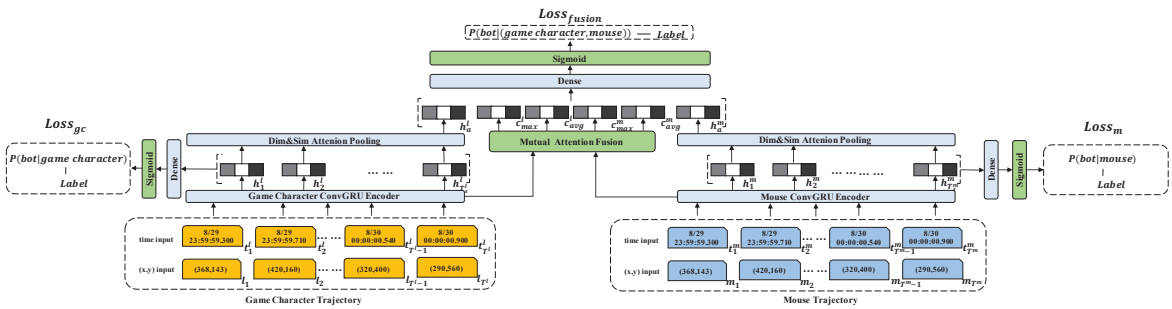

Fig. 6: Training process with TrilLoss.

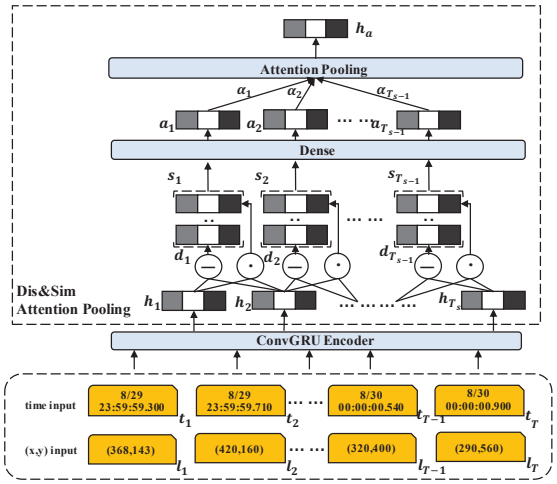

Fig. 7: Dis&Sim Attention Pooling

*2) **Mutul Attention–fusing two types of trajectories together:*** In order to capture the correlation between game character trajectories and mouse trajectories, we introduce a mutual attention module [5] to integrate the two types of trajectories. Meanwhile, this module achieves the soft alignment of the two trajectories. The time of generating two trajectories are not exactly aligned, since there are subtle differences in their time stamps. To be specific, we firstly compute the inner product of game character encoded sequences and mouse encoded sequences as the similarity matrix of the two sequences in Equation (7). Using the similarity matrix as weights, the mouse alignment sequence of game character $(\overline{\mathbf{h}}_1^l, \overline{\mathbf{h}}_2^l, \cdots, \overline{\mathbf{h}}_{T_s^l-1}^l, \overline{\mathbf{h}}_{T_s^l}^l)$ are obtained by the weighted sum of mouse encoded sequences, which is shown in Equation (8).

$$e_{ij} = \mathbf{h}_i^{l^T} \mathbf{h}_i^m \tag{7}$$

$$\overline{\mathbf{h}_i^l} = \sum_{j=1}^{T_s^m} \frac{\exp(e_{ij})}{\sum_{k=1}^{T_s^m} \exp(e_{ik})} \mathbf{h}_j^m, \forall i \in \left[1, \ldots, T_s^l\right] \tag{8}$$

$$\overline{\mathbf{h}_i^m} = \sum_{i=1}^{T_s^l} \frac{\exp(e_{ij})}{\sum_{k=1}^{T_s^l} \exp(e_{kj})} \overline{\mathbf{a}}_i, \forall j \in [1, \ldots, T_s^m] \tag{9}$$

where $\overline{\mathbf{h}_i^l}$ is the weighted sum of $(\mathbf{h}_1^m, \mathbf{h}_2^m, \cdots, \mathbf{h}_{T_s^m-1}^m, \mathbf{h}_{T_s^m}^m)$.

The game character alignment sequence of mouse is computed in the same way in Equation (9). Then, we compute the differences and Hadamard product between the aligned sequences and the output encoded sequence to further capture the difference and similarity of the two types of trajectories. Each hidden state in the sequences are concatenated together, shown in Equation (10). The concatenated sequences are input into another bidirectional GRUs to get the correlation sequences in Equation (11). Finally, the two correlation sequences are compressed into vectors with fixed dimensions via average pooling and max pooling, respectively, in Equation (12).

$$\begin{aligned} \mathbf{c}^l &= [\mathbf{h}^l; \overline{\mathbf{h}^l}; \mathbf{h}^l - \overline{\mathbf{h}^l}; \mathbf{h}^l \odot \overline{\mathbf{h}^l}] \\ \mathbf{c}^m &= [\mathbf{h}^m; \overline{\mathbf{h}^m}; \mathbf{h}^m - \overline{\mathbf{h}^m}; \mathbf{h}^m \odot \overline{\mathbf{h}^m}] \end{aligned} \tag{10}$$

$$\begin{aligned} \mathbf{o}_i^l &= \mathbf{BiGRU}\left(\mathbf{c}_i^l\right) \quad i = 1, 2, \ldots, T_{s_l} \\ \mathbf{o}_i^m &= \mathbf{BiGRU}\left(\mathbf{c}_i^m\right) \quad i = 1, 2, \ldots, T_{s_m} \end{aligned} \tag{11}$$

$$\begin{aligned} \mathbf{c}^l_{\text{ave}} &= \sum_{i=1}^{T_{s_l}} \frac{\mathbf{c}_i^l}{T_{s_l}}, \quad \mathbf{c}^l_{\max} = \max_{i=1}^{T_{s_l}} \mathbf{c}_i^l, \\ \mathbf{c}^m_{\text{ave}} &= \sum_{i=1}^{T_{s_m}} \frac{\mathbf{c}_i^m}{T_{s_m}}, \quad \mathbf{c}^m_{\max} = \max_{i=1}^{T_{s_m}} \mathbf{c}_i^m, \end{aligned} \tag{12}$$

*3) **TriLoss based training:*** The output of Mutual Attention module is concatenated together with the output vectors $\mathbf{h}_a^l, \mathbf{h}_a^m$ of Dis&Sim Pooling as the representation $\mathbf{c}$ for detecting bots. The probability of being a bot is computed in Equation (13). We compute the Binary Cross Entropy of the probability and the target label as l_f. In addition, we use the hidden state output by the game character encoder and the hidden state output by the mouse encoder to separately compute the bot probability p^{gc} and p^m to get the l_{gc} and l_m with the target label. They are summed with l_f to form our final loss in Equation (15). Thus, there are three loss, l_f, l_{gc}, and l_m, and we name it TriLoss.

The TriLoss enables our model can still accurately detect bots, even when a certain type of trajectories is missing. It makes the model performance robust, since trajectory data missing is unavoidable during collection process due to temporal failure, etc. TriLoss alleviates the problem of trajectory data missing through the loss of l_f, l_{gc}, and l_m. Specifically, l_f gives our model the ability to accurately detect the robot by combining the two types of trajectories, l_{gc} and l_m gives two encoders the ability to detect the robot singly.

$$\begin{aligned} \mathbf{c} &= \left[\mathbf{h}_a^l : \mathbf{c}^l_{\text{ave}} : \mathbf{c}^l_{\max} : \mathbf{h}_a^m : \mathbf{c}^m_{\text{ave}} : \mathbf{c}^m_{\max}\right] \\ p^f &= p\left(\text{bot} \mid \{l_i\}_{i=1}^{T^l}, \{m_i\}_{i=1}^{T^m}\right) \\ &= \text{sigmoid}(\boldsymbol{w} \cdot \mathbf{c} + b) \end{aligned} \tag{13}$$

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

$$
\begin{aligned}
p^{gc} &= p\left(\text{bot} \mid \{l_i\}_{i=1}^{T^l}\right) \\
&= \text{sigmoid}\left(\boldsymbol{w}\left[\mathbf{h}_1^l : \mathbf{h}_{T_s^l}^l\right] + b\right) \\
p^{m} &= p\left(\text{bot} \mid \{m_i\}_{i=1}^{T^m}\right) \\
&= \text{sigmoid}\left(\boldsymbol{w}\left[\mathbf{h}_1^m : \mathbf{h}_{T_s^m}^m\right] + b\right)
\end{aligned} \tag{14}
$$

$$
\begin{aligned}
l_f &= \frac{1}{N}\sum_{i=0}^{N} y_i \log p_i^f + (1-y_i)\log\left(1-p_i^f\right) \\
l_{gc} &= \frac{1}{N}\sum_{i=0}^{N} y_i \log p_i^{gc} + (1-y_i)\log\left(1-p_i^{gc}\right) \\
l_m &= \frac{1}{N}\sum_{i=0}^{N} y_i \log p_i^{m} + (1-y_i)\log\left(1-p_i^{m}\right) \\
l &= l_f + l_{gc} + l_m
\end{aligned} \tag{15}
$$

V. Experiment

In this section, we have conducted extensive experiments to show the effectiveness of T-Detector. We first describe the dataset used in the experiments.

A. Experiment Setup

*1) **Dataset Description**:* We evaluated our T-Detector with a large-scale real-world dataset collected by NetEase Inc., a Chinese Internet technology company. As one of the largest Internet and video game company, NetEase Inc. develops and operates many MMORPGs. The dataset we used contains game character trajectories and mouse trajectories of Justice [1], a MMORPG developed by NetEase Inc. [2]. Justice is a martial game about vicious manhunts based on a Wuxia novel of The Four Great Constables in the Northern Song era.

In the Justice dataset, there are 99,867 trajectory records from 26,539 users, during the time period from September 30th, 2020 to October 7th, 2020. There are 87,253 samples from human players and 12,614 bot samples. The collection of each record begins when one player (human or bot) enters a game map, and ends when the player exits the map. Each record consists of: 1) anonymized identification (ID) of one player (human or bot), 2) timestamp when the record begins to be collected, 3) game character trajectory in about 60-120 seconds, and 4) mouse trajectory collected in the same time period as the game character trajectory. Both game character trajectories and mouse trajectories are composed of a sequence of (x,y) coordinates. The sampling frequency is approximately 400 ms. Here, the game bots were manually identified and labeled by the NetEase Game Operation teams. We performed a statistic on the dataset, shown in Tab. I. There are some samples missing one type of trajectory, for example, there are 35,053 samples missing the game character trajectories.

In order to investigate the generalization ability of our T-Detector, we introduced the trajectories from another MMORPG that is also developed by NetEase Inc. It is anonymized by the cooperated company and we name the data as MMORPG data. It contains 4,4071 trajectory records from 6,770 human players and 1,512 bots, from May 10th, 2021 to May 16th, 2021. Its statistic is shown in Tab. I. Although the bot samples are fewer than human ones, there are still various behavior patterns among the bots. It may be because the bots are developed by different developers. The number of the behavior patterns of bot samples is close to that of humans.

It is worth noting that the bots in our dataset are labeled by experienced game operators in the company. The bot labels are with 100% precision. The game operators strictly identify the bots from the suspects complained by normal players, following strict procedures. They professionally confirm whether the suspects are bots from various cues, e.g., game logs, used devices, IP address, and game advantages.

The datasets do not contain any personally identifiable information, where the ”player ID” has been anonymized and does not contain any player metadata.

*2) **Performance metrics and Implementation**:* Considering there are relatively few bot samples, and we pay more attention to the detection results of bots, we used PR-AUC (the AUC of the precision-recall curves) to measure the performance of our model. F1-score is also used for measurement. In order to test the contribution of the pretraining part and the generation ability of our framework, we randomly selected only 5% of the samples for training, and others for testing. When implementing the T-Detector, we used AdamW optimizer, set learning rate as 0.005, dropout rate 0.2, batch size 32 for pretraining and fine tuning, 8 and 10 epochs for fine tuning and pretraining, temporal embedding and spatial embedding size 100, and improved CBOW window size as 20. All experiments were run on a machine with an Intel Xeon E5-2620 CPU and an Nvidia Titan xp 12G GPU.

*3) **Compared approaches**:* Motivated by [2], we selected the following algorithms for comparison: MLP (Multi-layer Perceptron), CNN, BiGRU. In order to compare with the baseline method, we tested the performance of ConvGRU, whose structure is the same as that of our ConvGRU encoder. In particular, all neural network algorithms adopt an embedding layer to embed the input trajectories and fine-tune the embedding layer in the training procedure. We also compared our model with traditional machine learning methods using statistical features, including GBDT (Gradient Boosting Decision Tree), SVM (Support Vector Machine) and LR (Logistic Regression). Moreover, we compared ours with [19] which used avatar trajectories, and two existing trajectory classification methods of MARC [18] and MSLNet [9].

(1) **MLP** [30] consists of two fully-connected layers, whose input is the concatenated vector of the average of multiple steps in a game character trajectory embedding sequence and a mouse trajectory embedding sequence.
(2) **CNN** [31] is composed of two 3-layer convolution modules for two types of trajectories. Their output are compressed as two hidden states with fixed dimensions by global average pooling, which are concatenated together and pass through a fully-connected layer to detect bots.

[1]https://leihuo.163.com/en/games.html?g=game-1nsh
[2]http://game.163.com

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Statistics of the two datasets.

Dataset	Users	Samples	Human Samples	Bot Samples	Samples without game character trajectory	Samples without mouse trajectory
Justice data	26,539	99,867	87,253	12,614	35,053	15,679
MMORPG data	6,770	44,071	42,559	1,512	7,671	3,448

TABLE II: PR-AUC comparison with other approaches on Justice data.

PR-AUC	Single Loss			TriLoss		
Model \ Embedding	No pretrain	Word2vec	LocationTime2Vec	No pretrain	Word2vec	LocationTime2Vec
MLP	0.8925	0.9275	0.9116	0.8926	0.9342	0.6016
CNN	0.8309	0.9126	0.9023	0.8655	0.9057	0.9200
BiGRU	0.9030	0.9217	0.9573	0.9132	0.9363	0.9652
ConvGRU	0.9191	0.9282	0.9531	0.9402	0.9405	0.9613
T-Detector	**0.9594**	**0.9464**	**0.9688**	**0.9556**	**0.9684**	**0.9777**

TABLE III: F1-score comparison with other approaches on Justice data.

F1	Single Loss			TriLoss		
Model \ Embedding	No pretrain	Word2vec	LocationTime2Vec	No pretrain	Word2vec	LocationTime2Vec
MLP	0.8397	0.8782	0.6029	0.8479	0.8775	0.8654
CNN	0.7994	0.8459	0.8356	0.8271	0.8634	0.8517
BiGRU	0.8582	0.8565	0.8934	0.8687	0.8889	0.9193
ConvGRU	0.8532	0.8621	0.8653	0.8878	0.8953	0.9172
T-Detector	**0.9141**	**0.8795**	**0.9290**	**0.9277**	**0.9249**	**0.9439**

(3) **BiGRU** [32] uses two Bidirectional GRUs to process the embedding sequences of two trajectories. The beginning hidden state and last hidden state of the output hidden state sequence will be concatenated as a representation hidden state of each trajectory. The two representation hidden states will be concatenated and pass through a fully-connected layer for bot detection.

(4) **ConvGRU** uses two ConvGRU encoders proposed by us to get the hidden state sequence of each trajectory. Similar as BiGRU, the begin hidden state and end hidden state of two hidden state sequences are all concatenated together and pass through a fully-connected layer to detect bots.

(5) **Traditional machine learning methods (GBDT, SVM, and LR)** classify bots or human players and we extract statistical features from trajectories to input them, including speed, acceleration, distance, and turning angle, and their max, mean, median, and quantile values.

(6) **Avatar detection in [19]** uses only the avatar trajectories to detect avatars with dissimilarity measurements.

(7) **MARC in [18]** is a trajectory classification method. It used geohash to convert x and y coordinate into fixed-dimensional vectors, which is then averaged with the time embedding vector and game map index embedding vector. The vector sequences are input into RNN for encoding, and the output vector of the last step of RNN is used for trajectory classification.

(8) **MSLNet in [9]** is a trajectory classification method for identifying transportation mode. It takes the grid map as the input, and transforms the trajectory into a point sequence. Then, DenseNet and Attention modules are applied for encoding the point sequences and classifying the transportation mode.

B. Results and Analysis

*1) **Comparison with other approaches**:* We compared T-Detector with MLP, CNN, BiGRU, and ConvGRU using Justice data, shown in Tab. II, and Tab. III. We tested both TriLoss and Single Loss in training procedure on all the approaches, where TriLoss is the l and Single Loss refers to the l_f in Equation (15). It is woth noting we also compared different embedding ways, including no pretraining, word2vec pretrain, and our LocationTime2Vec pretrain. As shown in Tab. II and Tab. III, it is found that:

TABLE IV: Comparison with traditional machine learning and existing trajectory classification methods on Justice data.

	PR-AUC	F1-score
GBDT	0.9340	0.8200
SVM	0.9097	0.7824
LR	0.9124	0.7696
Avatar Detection [19]	0.8137	0.6670
MARC [18]	0.7400	0.6710
MSLNet [9]	0.7939	0.7710
T-Detector	**0.9777**	**0.9439**

- Our T-Detector performs better than other approaches under all the experiment settings. When using LocationTime2Vec for embedding and TriLoss for training, the T-Detector achieves the best performance of PR-AUC 0.9777 and F1-score 0.9439. The performance is about 38% higher in PR-AUC and 8% higher in F1-score than MLP, and about 1% higher in PR-AUC and 2.5% higher in F1-score than BiGRU and ConvGRU. The comparison shows the advantages and effectiveness of T-Detector in feature representation, extraction and fusion. With LocationTime2Vec and TriLoss, MLP performs the worst in PR-AUC (0.6016), and perform similarly as CNN in F1-score (0.8654 v.s. 0.8517). BiGRU and ConvGRU obtain similar performance (AUC 0.96522 v.s. 0.9613, F1-score 0.9193 v.s. 0.9172). The CNN obtains the worst F1-score. It is because there is much redundancy in the trajectories and the valid movement usually occurs after many time steps. Limited by the size of receptive field, CNN cannot deal with the over-long distance dependence.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

TABLE V: PR-AUC comparison with other approaches on MMORPG data.

PR-AUC	Single Loss			TriLoss		
Model \ Embedding	No pretrain	Word2vec	LocationTime2Vec	No pretrain	Word2vec	LocationTime2Vec
MLP	0.8408	0.8939	0.9408	0.8555	0.8922	0.9338
CNN	0.8165	0.8827	0.9277	0.8186	0.8716	0.9269
BiGRU	0.8609	0.8951	0.9510	0.8610	0.9147	0.9528
ConvGRU	0.7640	0.8944	0.9255	0.8362	0.8880	0.9218
T-Detector	**0.8934**	**0.8977**	**0.9542**	**0.8883**	**0.9170**	**0.9608**

TABLE VI: F1-score comparison with other approaches on MMORPG data.

F1	Single Loss			TriLoss		
Model \ Embedding	No pretrain	Word2vec	LocationTime2Vec	No pretrain	Word2vec	LocationTime2Vec
MLP	0.802	0.8499	0.8779	0.8290	0.8533	0.8792
CNN	0.803	0.8421	0.8706	0.806	0.8405	0.8740
BiGRU	0.8141	0.8491	0.8982	0.8227	0.8572	0.9066
ConvGRU	0.7737	0.8212	0.8572	0.772	0.8350	0.8698
T-Detector	**0.8307**	**0.8644**	**0.9002**	**0.8505**	**0.8691**	**0.9110**

- The TriLoss based training contributes significantly to the bot detection performance on most models. For example, the performance of T-Detector with LocationTime2Vec improves from 0.9290 to 0.9439 in F1-score, when using Single Loss and TriLoss. The AUC of MLP with LocationTime2Vec improves a lot when using TriLoss (0.9737 v.s. 0.7948). It suggests that TriLoss based training benefits to the bot detection when there is only one type of trajectories, since there are many samples missing one type of trajectories in our Justice dataset.
- Of all the embedding ways, the LocationTime2Vec contributes the most to bot detection. For example, the PR-AUC of T-Detector with TriLoss gains from 0.9556 to 0.9777 when using LocationTime2Vec. However, the LocationTime2Vec does not work so well in MLP model, especially when training with TriLoss, while word2vec pretrain performs well for MLP. In contrast, LocationTime2Vec performs well when it is used in BiGRU and ConvGRU. It may be because the average operation of word2vec pretraining is similar to the average sequence embedding of MLP, which is helpful for training.
- We also compared the execution time of T-Detector with BiGRU and ConvGRU. T-Detector holds a distinct advantage over BiGRU in execution time. BiGRU takes approximately twice the execution time of T-Detector. The baseline method of ConvGRU takes the shortest time for detection, but its performance was the worst. Thus, T-Detector, not only with the best performance but also runs quickly, is preferred for deployment in practice.

In Tab. IV, we compared T-Detector with traditional machine learning methods (GBDT, SVM, and LR) that use statistical features (e.g., speed, acceleration, distance, and turning angle) as input. The performance is much worse than that of T-Detector, especially in F1-score. For example, the F1-score of LR is about zbout 18% lower than that of T-Detector (0.7696 v.s. 0.9439), and the PR-AUC is around 6% lower (0.9124 v.s. 0.9777). The statistical features cannot be generalized to other games, and they are not so powerful as behavioral features we used for bot detection. When compared with Avatar Detection [19] which only used avatar trajectories, T-Detector performs much better (PR-AUC 0.8137 v.s. 0.9777, F1 0.6670 v.s. 0.9439). It indicates the fusion of two trajectory types improves the performance. Moreover, the method in [19] is expensive in computation, and not so preferable when there are large amounts of test samples. MARC [18] and MSLNet [9] perform much worse than ours (PR-AUC 0.7400 v.s. 0.7939 v.s. 0.9777, F1 0.6710 v.s. 0.7710 v.s. 0.9439). It may because they focus on where to go and passby (e.g., a district), more coarse-grained, while we pay more attention to movement details (e.g., turning angel).

*2) **Generalization ability of T-Detector**:* We further investigated the generalization ability of T-Detector, to see whether it can work on other MMORPGs. To this end, we tested the performance of T-Detector using another MMORPG data, which is described in Section V-A1. We compared the performance of T-Detector with other approaches using the MMORPG data, shown in Tab. V and Tab. VI. As we can see, the T-Detector with TriLoss and LocationTime2Vec still performs better on the MMORPG data than all the other approaches, indicating the ability of being generalized to other MMORPGs. It achieves the best performance of 0.9608 in PR-AUC and 0.9110 in F1-score, very close to the best performance of T-Detector on Justice data (AUC 0.9608 v.s. 0.9777, F1-score 0.9110 v.s. 0.9439). The TriLoss and LocationTime2Vec still work well on this new dataset. Taken together, our T-Detector have the ability of being generalized to other MMORPGs, which is crucial to game companies.

*3) **Effectiveness of different modules**:* We investigated the effectiveness of the modules we designed in T-Detector, including LocationTime2Vec, Angle Pretrain, Dis&Sim Attention Pooling and Mutul Attention. More specifically, we tool the ConvGRU as the baseline where the embedding matrix is pre-trained using word2vec. We added each module to the baseline ConvGRU, and then compared PR-AUC and F1-score on Justice data, shown in Tab. VII. Besides, we also performed a Paired T test on the improvement gained by each module, to further show the effectiveness. From Tab. VII, it can be seen:

- **LocationTime2Vec** gets AUC 2.08% higher and F1-score 2.19% higher, respectively. It indicates that the initialization of embedding matrices plays an important role for

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

TABLE VII: Effectiveness of different modules on Justice data.

	PR-AUC	improvement	F1-score	improvement	p value
Baseline	0.9405	0%	0.8953	0%	0.00
Baseline+LocationTime2Vec	0.9613	2.08%	0.9172	2.19%	1.929e-76
Baseline+Mutual Attention Fusion	0.9530	1.25%	0.9062	1.09%	1.926e-10
Baseline+Dis&sim Attention Pooling	0.9570	1.65%	0.9154	2.01%	2.424e-29
Baseline+Angle Pretrain	0.9514	1.09%	0.9142	1.89%	7.040e-46
T-Detector(without Angle Pretrain)	0.9630	2.25%	0.9191	2.38%	2.9141e-40
T-Detector	0.9777	3.72%	0.9439	4.86%	3.476e-223

model training. Besides, the spatial and temporal information in trajectories embedded by LocationTime2Vec contributes a lot for bot detection.

- **Mutual attention** makes the performance 1.25% and 1.09% higher in AUC and F1-score, respectively. The correlation between game character trajectories and mouse trajectories helps to detect bots.
- **Dis&Sim Attention Pooling** contributes 1.65% and 2.01% gain in PR-AUC and F1-score, respectively. It suggests the features of speed and distance extracted by Dis&Sim Attention Pooling benefit to bot detection.
- **Angle Pretrain & T-Detector(without Angle Pretrain)**: When we added the Angle Pretrain to the baseline model, the PR-AUC and F1-score improves 1.09% and 1.89%, respectively. The advantage of Angle Pretrain is not fully exploited on the baseline model, since it is designed on the basis of the spatial and temporal information of LocationTime2Vec. To further investigate its contribution, we compared the performance of the T-Detector without Angle Pretrain and the complete T-Detector. When removing the Angle Pretrain, both the PR-AUC and F1-score decrease a lot, 1.47% and 2.48%, respectively. This obviously demonstrates the importance of the turning angle features extracted by Angle Pretrain.
- **T-Detector** integrates all the modules together and significantly improves the performance, 3.72% and 4.86% higher than the baseline model in PR-AUC and F1-score, respectively. By combining all the modules, T-Detector comprehensively considers the spatial and time information in trajectories, behavioral differences between human players and bots, and correlation between trajectories.
- **Paired T test**: The p values of all the comparison are much lower than 0.05, and we can accept all the hypothesis that the compared methods have a higher probability of classifying correctly on the test sets than the baseline model with a confidence of 95-99%.

*4) **Performance on different amounts of training data**:* To further explore the importance of the pretraining, we investigated the effect of varying number of training samples. We trained the models using 1%, 2%, 3%, 4%, and 5% amount of Justice samples, respectively, and used the 95% for test. The random selection of the training data was carried out 5 times independently, and the results were averaged. We compared the performance of the T-Detector without pretrain and the pre-trained T-Detector (LocationTime2Vec pretrain and Angle Pretrain), shown in Fig. 8. When using only 1% of samples for training, T-Detector performs much better than the T-Detector without pretrain (PR-AUC 0.9104 v.s. 0.7081, F1-score 0.8490 v.s. 0.7612), especially in PR-AUC, indicating pretraining is important for model training with a small limited dataset. The gap in performance between the two models shrinks as the amount of training samples increases. The pre-trained T-Detector still performs well when we used only 1% of the samples for training, achieving 0.9104 in AUC and 0.8490 in F1-score. The differences in performance become smaller when from 2% to 5% of samples for training. It indicates that T-Detector performs stably with varying number of training data, even for very small amount.

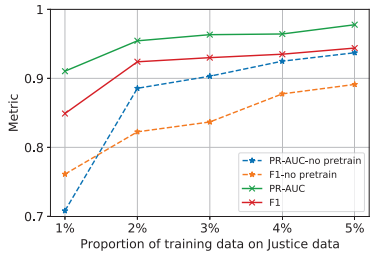

Fig. 8: Performance with different amount of Justice data (T-Detector and T-Detector without pretrain).

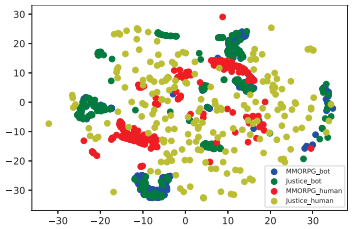

Fig. 9: t-SNE visualization of players.

*5) **Visualization of players**:* One obvious advantage of T-Detector is that it extract universal features rather than game-domain features, so that it can be generalized to many other MMORPGs. In order to illustrate this advantage, we used t-SNE transformation to visualize the player representation vectors from Justice data and MMORPG data, shown in Fig. 9. Here, the player representation vectors refer to the final concatenated representation vector c in Equation (13). As we can see from Fig. 9, game bots and human players are nicely separated, indicating T-Detector can well identify bots and humans. Moreover, the game bots from Justice data and MMORPG data are clustered together, even though they are two totally different games and the game bots are also different. It further proves that our T-Detector can capture universal features rather than game-domain features, enabling it to be generalized to many other MMORPGs.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion

Game bots seriously affect game experience of honest players and break the balance of game ecosystem. It is an urgent problem to be addressed. In this paper, we propose a pre-trained model named T-Detector to detect bots from game character trajectories and mouse trajectories. Leveraging a large number of unlabeled samples, we propose a pretraining method of LocationTime2Vec, to modeling the representation of trajectories. Moreover, we capture the universal features from trajectories by designing a Angle Pretrain and Attention pooling module, such as turning angles, moving speed and distances. Such features are independent on any specific game, so that T-Detector can be generalized to many other MMORPGs. Extensive experiments were conducted on two MMORPG datasets, which contain 143,398 samples. Compared with other approaches, T-Detector achieves the state-of-the-art performance and demonstrates the powerful generalization ability.

Although bots are becoming more and more human-like, it is hard for bots to exactly mimic some human behavioral patterns. Human players make much more irregularity and turns than bots [19]. For example, some natural and subtle shakes are generated when humans use mouses to control game characters. In comparison, bots are mostly characterized by straight paths. Taking advantages of such behavioral differences, our T-Detector is not easy to defeat. It is essential to fully learn the behavioral differences between humans and bots, to detect more bots accurately in the future.

Acknowledgment

This work was partly supported by Natural Science Foundation of China (No. 61802342, 61772460, 61802340, 61925603). Dr Runze Wu and Dr. Gang Pan are the corresponding authors.

References

[1] N. Ducheneaut and R. J. Moore, "The social side of gaming: a study of interaction patterns in a massively multiplayer online game," in *Proceedings of the 2004 ACM conference on Computer supported cooperative work*, 2004, pp. 360–369.

[2] J. Tao, J. Xu, L. Gong, Y. Li, C. Fan, and Z. Zhao, "Nguard: A game bot detection framework for netease mmorpgs," in *ACM SIGKDD 2018*, 2018, pp. 811–820.

[3] H. Kwon, A. Mohaisen, J. Woo, Y. Kim, E. Lee, and H. K. Kim, "Crime scene reconstruction: Online gold farming network analysis," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 3, pp. 544–556, 2016.

[4] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez, "Becaptcha-mouse: Synthetic mouse trajectories and improved bot detection," *arXiv preprint arXiv:2005.00890*, 2020.

[5] Q. Chen, X. Zhu, Z. Ling, S. Wei, H. Jiang, and D. Inkpen, "Enhanced lstm for natural language inference," *arXiv preprint arXiv:1609.06038*, 2016.

[6] J. Xu, Y. Luo, J. Tao, C. Fan, Z. Zhao, and J. Lu, "Nguard+ an attention-based game bot detection framework via player behavior sequences," *TKDD*, vol. 14, no. 6, pp. 1–24, 2020.

[7] M. L. Bernardi, M. Cimitile, F. Martinelli, and F. Mercaldo, "A time series classification approach to game bot detection," in *Proceedings of the 7th International Conference on Web Intelligence, Mining and Semantics*, 2017, pp. 1–11.

[8] A. R. Kang, S. H. Jeong, A. Mohaisen, and H. K. Kim, "Multimodal game bot detection using user behavioral characteristics," *SpringerPlus*, vol. 5, no. 1, pp. 1–19, 2016.

[9] R. Zhang, P. Xie, C. Wang, G. Liu, and S. Wan, "Classifying transportation mode and speed from trajectory data via deep multi-scale learning," *Computer Networks*, vol. 162, p. 106861, 2019.

[10] S. Dabiri, C.-T. Lu, K. Heaslip, and C. K. Reddy, "Semi-supervised deep learning approach for transportation mode identification using gps trajectory data," *TKDE*, vol. 32, no. 5, pp. 1010–1023, 2019.

[11] G. Pan, G. Qi, Z. Wu, D. Zhang, and S. Li, "Land-use classification using taxi gps traces," *TITS*, vol. 14, no. 1, pp. 113–123, 2012.

[12] J. Mao, J. Liu, C. Jin, and A. Zhou, "Feature grouping–based trajectory outlier detection over distributed streams," *TIST*, vol. 12, no. 2, pp. 1–23, 2021.

[13] Y. Zheng, Q. Li, Y. Chen, X. Xie, and W.-Y. Ma, "Understanding mobility based on gps data," in *Proceedings of the 10th international conference on Ubiquitous computing*, 2008, pp. 312–321.

[14] L. K. Sharma, O. P. Vyas, S. Schieder, and A. K. Akasapu, "Nearest neighbour classification for trajectory data," in *International Conference on Advances in Information and Communication Technologies*. Springer, 2010, pp. 180–185.

[15] A. S. Júnior, C. Renso, and S. Matwin, "Analytic: An active learning system for trajectory classification," *IEEE computer graphics and applications*, vol. 37, no. 5, pp. 28–39, 2017.

[16] A. Soleymani, J. Cachat, K. Robinson, S. Dodge, A. Kalueff, and R. Weibel, "Integrating cross-scale analysis in the spatial and temporal domains for classification of behavioral movement," *Journal of Spatial Information Science*, vol. 2014, no. 8, pp. 1–25, 2014.

[17] S. Dabiri and K. Heaslip, "Inferring transportation modes from gps trajectories using a convolutional neural network," *Transportation research part C: emerging technologies*, vol. 86, pp. 360–371, 2018.

[18] L. May Petry, C. Leite Da Silva, A. Esuli, C. Renso, and V. Bogorny, "Marc: a robust method for multiple-aspect trajectory classification via space, time, and semantic embeddings," *International Journal of Geographical Information Science*, vol. 34, no. 7, pp. 1428–1450, 2020.

[19] H.-K. Pao, K.-T. Chen, and H.-C. Chang, "Game bot detection via avatar trajectory analysis," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 2, no. 3, pp. 162–175, 2010.

[20] K.-T. Chen, H.-K. K. Pao, and H.-C. Chang, "Game bot identification based on manifold learning," in *Proceedings of the 7th ACM SIGCOMM Workshop on Network and System Support for Games*, 2008, pp. 21–26.

[21] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Science China Technological Sciences*, pp. 1–26, 2020.

[22] Y. Kim, "Convolutional neural networks for sentence classification. emnlp," 2014.

[23] B. Branco, P. Abreu, A. S. Gomes, M. S. Almeida, J. T. Ascensão, and P. Bizarro, "Interleaved sequence rnns for fraud detection," in *ACM SIGKDD 2020*, 2020, pp. 3101–3109.

[24] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *arXiv preprint arXiv:1310.4546*, 2013.

[25] S. Triberti, L. Milani, D. Villani, S. Grumi, S. Peracchia, G. Curcio, and G. Riva, "What matters is when you play: Investigating the relationship between online video games addiction and time spent playing over specific day phases," *Addictive Behaviors Reports*, vol. 8, pp. 185–188, 2018.

[26] Q. Ma, S. Muthukrishnan, and W. Simpson, "App2vec: Vector modeling of mobile apps and applications," in *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2016, pp. 599–606.

[27] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[28] W. Wang, Y. Huang, Y. Wang, and L. Wang, "Generalized autoencoder: A neural network framework for dimensionality reduction," in *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, 2014, pp. 490–497.

[29] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.

[30] F. Rosenblatt, *The perceptron, a perceiving and recognizing automaton Project Para*. Cornell Aeronautical Laboratory, 1957.

[31] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.

[32] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 19,2024 at 09:27:10 UTC from IEEE Xplore. Restrictions apply.